Name:- Shivasai Thodeti
Email:-shivasai591@gmail.com
Mobile:-6309342042
Batch Name:-B1_Python
Course:UGE_DoNet FSD (Python)
==========

1)STUDENT REGISTRATION LIST:
==========

```
<!DOCTYPE HTML>
<html>
<head>
   <title>Student Registration - jQuery</title>
   <!-- jQuery CDN -->
   <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
   <style>
      body {
         font-family: Arial, sans-serif;
      }
      input, button {
         padding: 8px;
         margin: 5px;
      }
      table {
         border-collapse: collapse;
         margin-top: 20px;
         width: 350px;
      }
      table, th, td {
         border: 1px solid black;
         padding: 8px;
         text-align: center;
```

```
        }
        tr:hover {
            background-color: #f2f2f2;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <h2>Student Registration Form</h2>
    <input type="text" id="name" placeholder="Enter Student Name">
    <input type="text" id="roll" placeholder="Enter Roll Number">
    <button id="addBtn">Add Student</button>
    <table id="studentTable">
        <tr>
            <th>Student Name</th>
            <th>Roll Number</th>
        </tr>
    </table>
    <script>
        $(document).ready(function () {
            // Add Student
            $("#addBtn").click(function () {
                let name = $("#name").val();
                let roll = $("#roll").val();
                if (name === "" || roll === "") {
                    alert("Please fill all fields");
                    return;
                }
                // Append new row
```

```
            $("#studentTable").append(
                "<tr><td>" + name + "</td><td>" + roll + "</td></tr>"
            );

            // Clear inputs
            $("#name").val("");
            $("#roll").val("");
        });

        // Remove row when clicked
        $("#studentTable").on("click", "tr", function () {

            // Avoid removing header row
            if ($(this).index() !== 0) {
                $(this).remove();
            }

        });

    });
  </script>

</body>
</html>
```

2) LIVE CHARACTER COUNT:

==========

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Live Character Counter</title>
    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

    <style>
        body {
            font-family: Arial, sans-serif;
        }
        textarea {
            width: 300px;
            height: 100px;
            padding: 10px;
        }
        #warning {
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h2>Live Character Counter</h2>
    <textarea id="message" placeholder="Type your message..."></textarea>
    <p>Character Count: <span id="count">0</span></p>
    <p id="warning"></p>
    <script>
        $(document).ready(function () {
            $("#message").keyup(function () {
                let textLength = $(this).val().length;
                // Update character count
                $("#count").text(textLength);
```

```
            // Check limit
            if (textLength > 100) {
                $("#warning")
                    .text("Limit Exceeded!")
                    .css("color", "red");
            } else {
                $("#warning")
                    .text("")
                    .css("color", "black");
            }
        });
    });
  </script>
</body>
</html>
```

3)CODE- SIMPLE SHOPPING CART ITEMS:

==========

```
<!DOCTYPE html>
<html>
<head>
  <title>Shopping Cart</title>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
    body {
      font-family: Arial, sans-serif;
    }
    select, button {
      padding: 8px;
```

```
            margin: 5px;
        }
        ul {
            margin-top: 20px;
        }
        li {
            margin: 5px 0;
        }
        .removeBtn {
            margin-left: 10px;
            color: white;
            background-color: red;
            border: none;
            padding: 4px 8px;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <h2>Simple Shopping Cart</h2>
    <!-- Dropdown -->
    <select id="items">
        <option value="">-- Select Item --</option>
        <option>Apple</option>
        <option>Mango</option>
        <option>Banana</option>
    </select>
    <button id="addBtn">Add to Cart</button>
    <h3>Cart Items:</h3>
```

```
<ul id="cartList"></ul>
<h4>Total Items: <span id="totalCount">0</span></h4>
<script>
   $(document).ready(function () {

      let selectedItem = "";
      // .change() → store selected item
      $("#items").change(function () {
         selectedItem = $(this).val();
      });
      // Add item to cart
      $("#addBtn").click(function () {
         if (selectedItem === "") {
            alert("Please select an item");
            return;
         }
         // .append() → add item with remove button
         $("#cartList").append(
            "<li>" + selectedItem +
            " <button class='removeBtn'>REMOVE</button></li>"
         );
         updateCount();
      });
      // .on() → remove item dynamically
      $("#cartList").on("click", ".removeBtn", function () {
         $(this).parent().remove();
         updateCount();
      });
```

```
        // .length() → count total items
        function updateCount() {
            let totalItems = $("#cartList li").length;
            $("#totalCount").text(totalItems);
        }
    });
  </script>
</body>
</html>
```

Date:-24/02/2026

1)Hospital Records to store Patient appointment Details

```
Drop tables if exist
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Doctors;
-- 1)Patients Table
CREATE TABLE Patients (
  PatientID INT PRIMARY KEY AUTO_INCREMENT,
  PatientName VARCHAR(100) NOT NULL,
  PatientPhone VARCHAR(15) NOT NULL UNIQUE
);
-- 2)Doctors Table
CREATE TABLE Doctors (
  DoctorID INT PRIMARY KEY AUTO_INCREMENT,
  DoctorName VARCHAR(100) NOT NULL,
  Specialization VARCHAR(100) NOT NULL
);
```

```
-- 3)Appointments Table
CREATE TABLE Appointments (
   AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
   PatientID INT NOT NULL,
   DoctorID INT NOT NULL,
   AppointmentDate DATE NOT NULL,
   Fees DECIMAL(8,2) NOT NULL,
   FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
   FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);
INSERT SAMPLE RECORDS :
INSERT INTO Patients (PatientName, PatientPhone) VALUES
('Reethu', '9876543210'),
('Kalpana', '9876543211'),
('Varsha', '9876543212'),
('Harini', '9876543213'),
('Yashu', '9876543214');
INSERT DOCTORS RECORDS:
INSERT INTO Doctors (DoctorName, Specialization) VALUES
('Dr. Reddy', 'Cardiologist'),
('Dr. Mehta', 'Dermatologist'),
('Dr. Iyer', 'Orthopedic');
INSERT APPOINTMENTS:
INSERT INTO Appointments (PatientID, DoctorID, AppointmentDate, Fees) VALUES
(1, 1, '2026-02-20', 800.00),
(2, 2, '2026-02-21', 500.00),
(3, 3, '2026-02-22', 700.00),
(4, 1, '2026-02-23', 800.00),
(5, 2, '2026-02-24', 500.00);
```

USING JOIN

```
SELECT
   p.PatientName,
   d.DoctorName,
   d.Specialization,
   a.AppointmentDate,
   a.Fees
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID;
```

2)CODE-COMPANY EMPLOYEES DETAILS:

CREATE TABLE WITH CONSTRAINTS:

Drop if exists

```
DROP TABLE IF EXISTS Projects;
DROP TABLE IF EXISTS Employees;
DROP TABLE IF EXISTS Departments;

--1) Departments Table
CREATE TABLE Departments (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(100) NOT NULL UNIQUE
);

-- 2) Employees Table
CREATE TABLE Employees (
   EmpID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Salary DECIMAL(10,2) NOT NULL CHECK (Salary > 0),
```

```
    DeptID INT NOT NULL,
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

-- 3)Projects Table
CREATE TABLE Projects (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(100) NOT NULL,
    DeptID INT NOT NULL,
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
INSERT SAMPLE DATA:
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');
INSERT EMPLOYEES:
INSERT INTO Employees VALUES
(101, 'Kalpana', 60000, 1),
(102, 'Reethu', 45000, 2),
(103, 'Varsha', 75000, 1),
(104, 'Harini', 52000, 3),
(105, 'Yashu’, 48000, 2);
INSERT PROJECTS:
INSERT INTO Projects VALUES
(201, 'Website Development', 1),
(202, 'Recruitment System', 2),
(203, 'Payroll System', 3),
(204, 'Mobile App', 1);
```

```
SELECT
   e.EmpID,
   e.Name,
   d.DeptName,
   p.ProjectName,
   (e.Salary * 12) AS AnnualSalary
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
JOIN Projects p ON d.DeptID = p.DeptID
WHERE e.Salary > 50000
ORDER BY e.Salary DESC;
(e.Salary * 12) AS AnnualSalary
```

3)CODE-A SHOPPING APP:

CREATE TABLE :

```
DROP TABLE IF EXISTS Orders;
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
   CustomerName VARCHAR(100) NOT NULL,
   ProductName VARCHAR(100) NOT NULL,
   Category VARCHAR(50) NOT NULL,
   Quantity INT NOT NULL CHECK (Quantity > 0),
   Price DECIMAL(10,2) NOT NULL CHECK (Price > 0)
);
```

INSERT SAMPLE RECORDS:

```
INSERT INTO Orders VALUES
(1, 'Harini', 'Laptop', 'Electronics', 1, 55000),
(2, 'Kalpana’, 'T-Shirt', 'Clothing', 3, 800),
(3, 'Varsha', 'Mobile', 'Electronics', 1, 30000),
```

```
(4, 'Reethu', 'Rice Bag', 'Grocery', 5, 600),
(5, 'Yashu', 'Headphones', 'Electronics', 2, 2000),
(6, 'Shahi, 'Jeans', 'Clothing', 2, 1500),
(7, 'Harsha', 'Vegetables', 'Grocery', 10, 100),
(8, 'Wasima', 'TV', 'Electronics', 1, 40000);
```

CALCULATE TOTAL AMOUNT FOR EACH ORDER :

```
SELECT
   OrderID,
   CustomerName,
   Category,
   (Quantity * Price) AS TotalAmount
FROM Orders;
```

CATEGORY WITH HIGHEST TOTAL SALES:

```
SELECT Category,
      SUM(Quantity * Price) AS TotalSales
FROM Orders
GROUP BY Category
ORDER BY TotalSales DESC
LIMIT 1;
```

CUSTOMERS WHO SPENT MORE THAN AVERAGE :

```
SELECT CustomerName,
      SUM(Quantity * Price) AS TotalSpent
FROM Orders
GROUP BY CustomerName
HAVING SUM(Quantity * Price) >
(
   SELECT AVG(CustomerTotal)
   FROM (
      SELECT SUM(Quantity * Price) AS CustomerTotal
```

```
        FROM Orders
        GROUP BY CustomerName
    ) AS AvgTable
);
```

CATEGORY-WISE TOTAL SALES :

```
SELECT Category,
       SUM(Quantity * Price) AS TotalSales
FROM Orders
GROUP BY Category
HAVING SUM(Quantity * Price) > 5000;
```

4)TRAINING INSTITUTE BATCHES :

CREATE BATCH TABLES :

```
DROP TABLE IF EXISTS BatchA;
DROP TABLE IF EXISTS BatchB;
CREATE TABLE BatchA (
    StudentName VARCHAR(100) NOT NULL
);
CREATE TABLE BatchB (
    StudentName VARCHAR(100) NOT NULL
);
```

INSERT STUDENTS :

```
-- Batch A (6 students)
INSERT INTO BatchA VALUES
('shivasai'),
('Harini'),
('Varsha'),
('Reethu'),
('Yashu'),
('Shahi');
```

```
-- Batch B (6 students, 2 common: shivasai, Harini)
INSERT INTO BatchB VALUES
('shivasai'),
('Harini'),
('Wasima'),
('Manisha'),
('Kavya'),
('Charitha');
STUDENTS PRESENT IN BOTH BATCHES :
SELECT StudentName FROM BatchA
INTERSECT
SELECT StudentName FROM BatchB;
ALL STUDENTS :
SELECT StudentName FROM BatchA
UNION
SELECT StudentName FROM BatchB;
STUDENTS ONLY IN BATCH A :
SELECT StudentName FROM BatchA
EXCEPT
SELECT StudentName FROM BatchB;
CREATE FEES DETAILS :
DROP TABLE IF EXISTS Fees;

CREATE TABLE Fees (
   StudentName VARCHAR(100) NOT NULL,
   FeesPaid DECIMAL(10,2) NOT NULL CHECK (FeesPaid >= 0),
   CourseName VARCHAR(100) NOT NULL
);
INSERT FEES DETAILS :
```

```
INSERT INTO Fees VALUES
('shivasai’, 5000, 'SQL'),
('Harini', 1500, 'SQL'),
('Varsha', 2500, 'Python'),
('Reethu', 4000, 'SQL'),
('Yashu’, 1800, 'Java'),
('Shahi', 3500, 'SQL');
```

UPDATE :

```
UPDATE Fees
SET FeesPaid = FeesPaid + 1000
WHERE CourseName = 'SQL';
```

DELETE :

```
DELETE FROM Fees
WHERE FeesPaid < 2000;
```

CASE EXPRESSION :

```
SELECT
   StudentName,
   FeesPaid,
   CASE
      WHEN FeesPaid >= 3000 THEN 'Paid'
      ELSE 'Due'
   END AS Status
FROM Fees;
```

5): E- COMMERCE ORDER INTEGRITY :

CREATE TABLE WITH CONSTRAINTS :

```
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Categories;
```

```
DROP TABLE IF EXISTS Customers;

-- 1)Customers
CREATE TABLE Customers (
   CustomerID INT PRIMARY KEY AUTO_INCREMENT,
   CustomerName VARCHAR(100) NOT NULL,
   CustomerEmail VARCHAR(150) NOT NULL UNIQUE
);

-- 2)Categories
CREATE TABLE Categories (
   CategoryID INT PRIMARY KEY AUTO_INCREMENT,
   CategoryName VARCHAR(100) NOT NULL UNIQUE
);

-- 3)Products
CREATE TABLE Products (
   ProductID INT PRIMARY KEY AUTO_INCREMENT,
   ProductName VARCHAR(100) NOT NULL,
   CategoryID INT NOT NULL,
   Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
   FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);

-- 4)Orders
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY AUTO_INCREMENT,
   CustomerID INT NOT NULL,
   OrderDate DATE NOT NULL,
```

```
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

-- 5)OrderDetails (Bridge Table)
CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
INSERT SAMPLE DATA :
INSERT INTO Categories (CategoryName) VALUES
('Electronics'),
('Clothing'),
('Grocery');
INSERT CUSTOMERS :
INSERT INTO Customers (CustomerName, CustomerEmail) VALUES
('shivasai', 'shivasai591@gmail.com'),
('Harini', 'harini@gmail.com'),
('Reethu', 'reethu@gmail.com'),
('Varsha', 'varsha@gmail.com'),
('Yashu', 'yashu@gmail.com');
INSERT PRODUCTS :
INSERT INTO Products (ProductName, CategoryID, Price) VALUES
('Laptop', 1, 60000),
('Mobile', 1, 30000),
('T-Shirt', 2, 800),
```

('Rice Bag', 3, 1200),

('Headphones', 1, 2000);

INSERT ORDERS :

INSERT INTO Orders (CustomerID, OrderDate) VALUES

(1, '2026-02-01'),

(2, '2026-02-02'),

(3, '2026-02-03'),

(4, '2026-02-04'),

(5, '2026-02-05'),

(1, '2026-02-06'),

(2, '2026-02-07'),

(3, '2026-02-08');

INSERT ORDER DETAILS :

INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES

(1, 1, 1),

(2, 3, 2),

(3, 2, 1),

(4, 4, 3),

(5, 5, 2),

(6, 3, 1),

(7, 1, 1),

(8, 4, 2);

MULTI JOIN :

```
SELECT
    c.CustomerName,
    p.ProductName,
    cat.CategoryName,
    od.Quantity,
    (od.Quantity * p.Price) AS TotalAmount
```

```
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories cat ON p.CategoryID = cat.CategoryID;
SELECT c.CustomerName
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY c.CustomerID
HAVING COUNT(DISTINCT p.CategoryID) > 1;
```

MOST SOLD PRODUCT :

```
SELECT p.ProductName,
      SUM(od.Quantity) AS TotalSold
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID
ORDER BY TotalSold DESC
LIMIT 1;
```

DELETE PRODUCTS NEVER ORDERED :

```
DELETE FROM Products
WHERE ProductID NOT IN (
   SELECT DISTINCT ProductID FROM OrderDetails
);
```

6)BANKING SYSTEM :

CREATE TABLE WITH CONSTRAINTS :

```
DROP TABLE IF EXISTS Transactions;
DROP TABLE IF EXISTS Accounts;
```

```
DROP TABLE IF EXISTS Customers;

-- 1)Customers
CREATE TABLE Customers (
   CustomerID INT PRIMARY KEY AUTO_INCREMENT,
   Name VARCHAR(100) NOT NULL,
   Phone VARCHAR(15) NOT NULL UNIQUE
);

-- 2)Accounts
CREATE TABLE Accounts (
   AccountID INT PRIMARY KEY AUTO_INCREMENT,
   CustomerID INT NOT NULL,
   Balance DECIMAL(12,2) NOT NULL CHECK (Balance >= 0),
   FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

-- 3)Transactions
CREATE TABLE Transactions (
   TransactionID INT PRIMARY KEY AUTO_INCREMENT,
   AccountID INT NOT NULL,
   Type VARCHAR(10) NOT NULL CHECK (Type IN ('Credit','Debit')),
   Amount DECIMAL(12,2) NOT NULL CHECK (Amount > 0),
   Date DATE NOT NULL,
   FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);
INSERT DATA :
INSERT INTO Customers (Name, Phone) VALUES
('Kalpana', '9876543210'),
```

('Harini', '9876543211'),

('Reethu', '9876543212'),

('Varsha', '9876543213'),

('Yashu', '9876543214');

INSERT 6 ACCOUNTS :

INSERT INTO Accounts (CustomerID, Balance) VALUES

(1, 120000),

(1, 50000),

(2, 80000),

(3, 150000),

(4, 20000),

(5, 0);

INSERT 10 TRANSACTIONS :

INSERT INTO Transactions (AccountID, Type, Amount, Date) VALUES

(1, 'Credit', 20000, '2026-02-01'),

(1, 'Debit', 10000, '2026-02-02'),

(2, 'Credit', 15000, '2026-02-03'),

(2, 'Debit', 5000, '2026-02-04'),

(3, 'Credit', 30000, '2026-02-05'),

(3, 'Debit', 10000, '2026-02-06'),

(4, 'Credit', 40000, '2026-02-07'),

(4, 'Debit', 20000, '2026-02-08'),

(5, 'Debit', 5000, '2026-02-09'),

(1, 'Credit', 25000, '2026-02-10');

USING CASE + GROUP BY :

```
SELECT
   c.Name AS CustomerName,
   a.AccountID,
   SUM(CASE WHEN t.Type = 'Credit' THEN t.Amount ELSE 0 END) AS TotalCredits,
```

```
    SUM(CASE WHEN t.Type = 'Debit' THEN t.Amount ELSE 0 END) AS TotalDebits,
    a.Balance AS CurrentBalance
FROM Accounts a
JOIN Customers c ON a.CustomerID = c.CustomerID
LEFT JOIN Transactions t ON a.AccountID = t.AccountID
GROUP BY c.Name, a.AccountID, a.Balance;
```

CUSTOMERS WITH TRANSACTIONS ABOVE AVERAGE :

```
SELECT DISTINCT c.Name
FROM Customers c
JOIN Accounts a ON c.CustomerID = a.CustomerID
JOIN Transactions t ON a.AccountID = t.AccountID
WHERE t.Amount > (
    SELECT AVG(Amount) FROM Transactions
);
```

BONUS :

```
UPDATE Accounts
SET Balance = Balance + (Balance * 0.02)
WHERE Balance > 100000;
```

DELETE ACCOUNTS WITH ZERO BALANCE :

```
DELETE FROM Accounts a
WHERE a.Balance = 0
AND NOT EXISTS (
    SELECT 1
    FROM Transactions t
    WHERE t.AccountID = a.AccountID
);
```

7) UNIVERSITY GRADING SYSTEM :

CREATE TABLES :

```
DROP TABLE IF EXISTS Marks;
```

```
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Departments;

-- 1)Departments
CREATE TABLE Departments (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(100) NOT NULL UNIQUE
);

-- 2)Students
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   DeptID INT NOT NULL,
   FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

-- 3)Marks
CREATE TABLE Marks (
   StudentID INT NOT NULL,
   Subject VARCHAR(100) NOT NULL,
   Marks INT NOT NULL CHECK (Marks BETWEEN 0 AND 100),
   FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
INSERT DEPARTMENTS:
INSERT INTO Departments VALUES
(1, 'Computer Science'),
(2, 'Mechanical'),
(3, 'Commerce');
```

INSERT STUDENTS :

```
INSERT INTO Students VALUES
(101, 'Rahul', 1),
(102, 'Anita', 1),
(103, 'Kiran', 2),
(104, 'Meena', 2),
(105, 'Suresh', 3),
(106, 'Priya', 3),
(107, 'Arjun', 1),
(108, 'Neha', 2);
```

INSERT MARKS :

```
INSERT INTO Marks VALUES
(101, 'Math', 95),
(101, 'DBMS', 88),

(102, 'Math', 72),
(102, 'DBMS', 78),

(103, 'Thermodynamics', 65),
(103, 'Mechanics', 70),

(104, 'Thermodynamics', 55),
(104, 'Mechanics', 60),

(105, 'Accounts', 85),
(105, 'Economics', 90),

(106, 'Accounts', 45),
(106, 'Economics', 50),
```

```
(107, 'Math', 92),
(108, 'Mechanics', 75);
TOTAL MARKS PER STUDENTS + GRADE (CASE) :
SELECT
   s.StudentID,
   s.Name,
   d.DeptName,
   SUM(m.Marks) AS TotalMarks,
   CASE
      WHEN SUM(m.Marks) >= 90 THEN 'A'
      WHEN SUM(m.Marks) >= 75 THEN 'B'
      WHEN SUM(m.Marks) >= 60 THEN 'C'
      ELSE 'Fail'
   END AS Grade
FROM Students s
JOIN Departments d ON s.DeptID = d.DeptID
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name, d.DeptName;
DEPARTMENT WITH HIGHEST AVERAGE MARKS :
SELECT DeptName,
      AVG(StudentTotal) AS DeptAverage
FROM (
   SELECT s.StudentID, s.DeptID,
         SUM(m.Marks) AS StudentTotal
   FROM Students s
   JOIN Marks m ON s.StudentID = m.StudentID
   GROUP BY s.StudentID, s.DeptID
) AS StudentTotals
```

```
JOIN Departments d ON StudentTotals.DeptID = d.DeptID
GROUP BY DeptName
ORDER BY DeptAverage DESC
LIMIT 1;
```

STUDENTS SCORING ABOVE THEIR DEPARTMENTS :

```
SELECT s.StudentID, s.Name
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name, s.DeptID
HAVING SUM(m.Marks) >
(
   SELECT AVG(TotalMarks)
   FROM (
      SELECT s2.StudentID,
           SUM(m2.Marks) AS TotalMarks
      FROM Students s2
      JOIN Marks m2 ON s2.StudentID = m2.StudentID
      WHERE s2.DeptID = s.DeptID
      GROUP BY s2.StudentID
   ) AS DeptAvg
);
```

DEPARTMENTS WHERE AVERAGE MARKS > 70 :

```
SELECT d.DeptName,
     AVG(StudentTotal) AS DeptAverage
FROM (
   SELECT s.StudentID, s.DeptID,
        SUM(m.Marks) AS StudentTotal
   FROM Students s
   JOIN Marks m ON s.StudentID = m.StudentID
```

```
    GROUP BY s.StudentID, s.DeptID
) AS StudentTotals
JOIN Departments d ON StudentTotals.DeptID = d.DeptID
GROUP BY d.DeptName
HAVING AVG(StudentTotal) > 70;
```

8)CODE- INVENTORY + SUPPLIER SYSTEM :

CREATE TABLES :

```
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Suppliers;

-- 1)Suppliers
CREATE TABLE Suppliers (
    SupplierID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

--  Products
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    SupplierID INT,
    StockQty INT NOT NULL CHECK (StockQty >= 0),
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID)
);

-- Orders
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
```

```
    ProductID INT NOT NULL,
    QuantityOrdered INT NOT NULL CHECK (QuantityOrdered > 0),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

INSERT SAMPLE DATA :

```
INSERT INTO Suppliers VALUES
(1, 'ABC Traders'),
(2, 'Global Supplies'),
(3, 'Fresh Mart'),
(4, 'TechSource'),
(5, 'Unused Supplier');
```

PRODUCTS :

```
INSERT INTO Products VALUES
(101, 'Laptop', 4, 300),
(102, 'Mobile', 4, 150),
(103, 'Rice Bag', 3, 600),
(104, 'Wheat Bag', 3, 180),
(105, 'Keyboard', 1, 50);
```

ORDERS :

```
INSERT INTO Orders VALUES
(1, 101, 200),
(2, 102, 180),
(3, 103, 100),
(4, 104, 250),
(5, 105, 60);
```

TASK – 1 :

```
SELECT s.SupplierID, s.Name
FROM Suppliers s
JOIN Products p ON s.SupplierID = p.SupplierID
```

```
GROUP BY s.SupplierID, s.Name
HAVING COUNT(p.ProductID) > 1;
```

TASK – 2 :

```
SELECT p.ProductID, p.Name, p.StockQty,
    SUM(o.QuantityOrdered) AS TotalOrdered
FROM Products p
JOIN Orders o ON p.ProductID = o.ProductID
GROUP BY p.ProductID, p.Name, p.StockQty
HAVING p.StockQty < SUM(o.QuantityOrdered);
```

TASK – 3 :

```
SELECT s.SupplierID, s.Name
FROM Suppliers s
LEFT JOIN Products p ON s.SupplierID = p.SupplierID
WHERE p.ProductID IS NULL;
```

TASK – 4 :

```
UPDATE Products
SET StockQty = StockQty + 50
WHERE StockQty < 100;
```

TASK – 5 :

```
DELETE FROM Suppliers s
WHERE NOT EXISTS (
  SELECT 1
  FROM Products p
  WHERE p.SupplierID = s.SupplierID
);
```

TASK – 6 :

```
SELECT
  ProductID,
  Name,
```

```
    StockQty,
    CASE
        WHEN StockQty >= 500 THEN 'High Stock'
        WHEN StockQty BETWEEN 200 AND 499 THEN 'Medium Stock'
        ELSE 'Low Stock'
    END AS StockStatus
FROM Products;
```

9)CODE :

CREATE TABLE WITH CONSTRAINTS :

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    SubscriptionType VARCHAR(20) DEFAULT 'Basic'
);

CREATE TABLE Movies (
    MovieID INT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    Genre VARCHAR(50) NOT NULL
);

CREATE TABLE WatchHistory (
    UserID INT,
    MovieID INT,
    WatchTime INT CHECK (WatchTime > 0), -- in minutes
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID)
);
```

INSERT SAMPLE DATA :

```
-- Users
INSERT INTO Users VALUES
(1, 'Kunal', 'Basic'),
(2, 'Ravi', 'Standard'),
(3, 'Sneha', 'Basic'),
(4, 'Anjali', 'Standard'),
(5, 'Arjun', 'Basic');

-- Movies
INSERT INTO Movies VALUES
(101, 'Inception', 'Sci-Fi'),
(102, 'Avengers', 'Action'),
(103, 'Titanic', 'Romance'),
(104, 'Interstellar', 'Sci-Fi'),
(105, 'Joker', 'Drama');

-- Watch History
INSERT INTO WatchHistory VALUES
(1, 101, 300),
(1, 102, 200),
(2, 101, 150),
(2, 103, 250),
(3, 104, 400),
(3, 105, 100),
(4, 102, 350),
(4, 103, 200),
(5, 105, 120),
(5, 101, 180);
```

TASK – 1 :

```
SELECT m.Title, SUM(w.WatchTime) AS TotalWatchTime
FROM Movies m
JOIN WatchHistory w ON m.MovieID = w.MovieID
GROUP BY m.Title
ORDER BY TotalWatchTime DESC
LIMIT 1;
```

TASK – 2 :

```
SELECT u.UserID, u.Name
FROM Users u
JOIN WatchHistory w ON u.UserID = w.UserID
JOIN Movies m ON w.MovieID = m.MovieID
GROUP BY u.UserID, u.Name
HAVING COUNT(DISTINCT m.Genre) > 2;
```

TASK – 3 :

```
SELECT m.Genre, SUM(w.WatchTime) AS TotalWatchTime
FROM Movies m
JOIN WatchHistory w ON m.MovieID = w.MovieID
GROUP BY m.Genre;
```

TASK – 4 :

```
UPDATE Users
SET SubscriptionType = 'Premium'
WHERE UserID IN (
   SELECT UserID
   FROM WatchHistory
   GROUP BY UserID
   HAVING SUM(WatchTime) >
      (SELECT AVG(UserTotal)
       FROM (
           SELECT SUM(WatchTime) AS UserTotal
```

```
            FROM WatchHistory
            GROUP BY UserID
        ) AS AvgTable)
);
```

TASK – 5 :

```
SELECT m.MovieID, m.Title
FROM Movies m
WHERE NOT EXISTS (
   SELECT 1
   FROM WatchHistory w
   WHERE w.MovieID = m.MovieID
);
```

10) HR PROMOTION LOGIC :

```
CREATE TABLE Departments (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
   EmpID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Salary DECIMAL(10,2) CHECK (Salary > 0),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

CREATE TABLE Performance (
   EmpID INT PRIMARY KEY,
   Rating INT CHECK (Rating BETWEEN 1 AND 5),
```

```
    FOREIGN KEY (EmpID) REFERENCES Employees(EmpID)
);
```

INSERT SAMPLE DATA :

```
-- Departments
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');

-- Employees
INSERT INTO Employees VALUES
(101, 'Kunal', 60000, 1),
(102, 'Ravi', 45000, 1),
(103, 'Sneha', 70000, 2),
(104, 'Anjali', 50000, 2),
(105, 'Arjun', 80000, 3),
(106, 'Meera', 30000, 3);

-- Performance
INSERT INTO Performance VALUES
(101, 5),
(102, 3),
(103, 4),
(104, 2),
(105, 5),
(106, 1);
```

TASK – 1 :

```
UPDATE Employees
SET Salary = Salary * 1.10
```

```
WHERE EmpID IN (
   SELECT EmpID
   FROM Performance
   WHERE Rating >= 4
);
```

TASK – 2 :

```
SELECT e.Name,
      d.DeptName,
      e.Salary,
      p.Rating,
      CASE
         WHEN p.Rating = 5 THEN 'Excellent'
         WHEN p.Rating = 4 THEN 'Good'
         WHEN p.Rating = 3 THEN 'Average'
         ELSE 'Poor'
      END AS PerformanceLevel
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
JOIN Performance p ON e.EmpID = p.EmpID;
```

TASK -3 :

```
SELECT d.DeptName,
      AVG(e.Salary) AS AvgSalary
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
GROUP BY d.DeptName
ORDER BY AvgSalary DESC
LIMIT 1;
```

TASK – 4 :

```
SELECT e.Name, e.Salary, d.DeptName
```

```
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
WHERE e.Salary > (
   SELECT AVG(e2.Salary)
   FROM Employees e2
   WHERE e2.DeptID = e.DeptID
);
```

TASK – 5 :

```
DELETE FROM Employees
WHERE EmpID IN (
   SELECT EmpID
   FROM Performanc
   WHERE Rating < 2
);
```

11) FRAUD DETECTION LOGIC :

CREATE TABLE :

```
CREATE TABLE Transactions (
   TransactionID INT PRIMARY KEY,
   UserID INT NOT NULL,
   Amount DECIMAL(12,2) CHECK (Amount > 0),
   Location VARCHAR(100),
   Date DATE
);
```

SAMPLE DATA :

```
INSERT INTO Transactions VALUES
(1, 101, 50000, 'Hyderabad', '2025-01-10'),
(2, 101, 120000, 'Hyderabad', '2025-01-10'),
(3, 101, 30000, 'Hyderabad', '2025-01-10'),
(4, 101, 20000, 'Hyderabad', '2025-01-10'),
```

```
(5, 102, 15000, 'Chennai', '2025-02-11'),
(6, 102, 25000, 'Bangalore', '2025-02-11'),
(7, 103, 200000, 'Mumbai', '2024-12-01'),
(8, 104, 8000, 'Delhi', '2020-03-15');
```

TASK – 1 :

```
SELECT UserID, Date, COUNT(*) AS TransactionCount
FROM Transactions
GROUP BY UserID, Date
HAVING COUNT(*) > 3;
```

TASK -2 :

```
SELECT UserID, COUNT(DISTINCT Location) AS LocationCount
FROM Transactions
GROUP BY UserID
HAVING COUNT(DISTINCT Location) > 1;
```

TASK – 3 :

```
SELECT t.TransactionID,
    t.UserID,
    t.Amount,
    t.Location,
    t.Date,
    CASE
        WHEN t.Amount > 100000 THEN 'High Risk'
        WHEN t.UserID IN (
            SELECT UserID
            FROM Transactions
            GROUP BY UserID
            HAVING COUNT(DISTINCT Location) > 1
        ) THEN 'Location Risk'
        ELSE 'Normal'
```

```
    END AS RiskStatus
FROM Transactions t;
```

TASK – 4 :

```
DELETE FROM Transactions
WHERE Date < CURDATE() - INTERVAL 5 YEAR;
DELETE FROM Transactions
WHERE Date < DATEADD(YEAR, -5, GETDATE());
```

Date:- 25/02/2026

PROJECT 1 — Smart Hospital Management & Billing Analytics System

DATA BASE DESIGNS : PATIENTS :

```
CREATE TABLE Patients (
   PatientID INT PRIMARY KEY,
   PatientName VARCHAR(100) NOT NULL,
   Phone VARCHAR(15) UNIQUE,
   Email VARCHAR(100) UNIQUE,
   DateOfBirth DATE,
   CreatedAt DATE DEFAULT CURRENT_DATE
);
```

2. SPECIALIZATIONS :

```
CREATE TABLE Specializations (
   SpecializationID INT PRIMARY KEY,
   SpecializationName VARCHAR(100) NOT NULL
);
```

3. DOCTORS :

```
CREATE TABLE Doctors (
   DoctorID INT PRIMARY KEY,
   DoctorName VARCHAR(100) NOT NULL,
   SpecializationID INT,
```

```
    Salary DECIMAL(12,2),
    FOREIGN KEY (SpecializationID) REFERENCES Specializations(SpecializationID)
);
```

4. APPOINTMENTS :

```
CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY,
    PatientID INT,
    DoctorID INT,
    AppointmentDate DATE NOT NULL,
    Status VARCHAR(20),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);
```

5. TREATMENTS :

```
CREATE TABLE Treatments (
    TreatmentID INT PRIMARY KEY,
    TreatmentName VARCHAR(100) NOT NULL,
    Cost DECIMAL(10,2) CHECK (Cost > 0)
);
```

6. INSURANCE PROVIDERS :

```
CREATE TABLE InsuranceProviders (
    InsuranceID INT PRIMARY KEY,
    ProviderName VARCHAR(100) NOT NULL,
    CoveragePercent DECIMAL(5,2)
);
```

7. BILLS :

```
CREATE TABLE Bills (
    BillID INT PRIMARY KEY,
    AppointmentID INT,
```

```
    TreatmentID INT,
    InsuranceID INT,
    BillAmount DECIMAL(12,2) CHECK (BillAmount > 0),
    InsuranceCoveredAmount DECIMAL(12,2) DEFAULT 0,
    PaymentStatus VARCHAR(20),
    BillDate DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID),
    FOREIGN KEY (InsuranceID) REFERENCES InsuranceProviders(InsuranceID)
);
```

A. Patient Billing Report :

```
SELECT
    p.PatientName,
    d.DoctorName,
    s.SpecializationName,
    t.TreatmentName,
    a.AppointmentDate,
    b.BillAmount,
    b.InsuranceCoveredAmount,
    (b.BillAmount - b.InsuranceCoveredAmount) AS FinalPayableAmount
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecializationID = s.SpecializationID
JOIN Treatments t ON b.TreatmentID = t.TreatmentID;
```

B. Top 3 Revenue Generating Doctors:

```
SELECT d.DoctorName,
      SUM(b.BillAmount) AS TotalRevenue
```

```
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Doctors d ON a.DoctorID = d.DoctorID
GROUP BY d.DoctorName
ORDER BY TotalRevenue DESC
LIMIT 3;
```

C. Identify Patients with Pending Payments:

```
SELECT
   p.PatientName,
   (b.BillAmount - b.InsuranceCoveredAmount) AS FinalPayableAmount,
   CASE
      WHEN (b.BillAmount - b.InsuranceCoveredAmount) > 0 THEN 'Pending'
      ELSE 'Paid'
   END AS PaymentStatus
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID;
```

D. Specializations Generating Revenue Above Average :

```
SELECT s.SpecializationName,
     SUM(b.BillAmount) AS TotalRevenue
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecializationID = s.SpecializationID
GROUP BY s.SpecializationName
HAVING SUM(b.BillAmount) >
    (SELECT AVG(TotalRevenue)
     FROM (
        SELECT SUM(b2.BillAmount) AS TotalRevenue
```

```
        FROM Bills b2
        JOIN Appointments a2 ON b2.AppointmentID = a2.AppointmentID
        JOIN Doctors d2 ON a2.DoctorID = d2.DoctorID
        GROUP BY d2.SpecializationID
    ) AS RevenueTable);
```

E. Delete Inactive Patients (No Appointments in Last 3 Years) :

```
DELETE FROM Patients p
WHERE NOT EXISTS (
  SELECT 1
  FROM Appointments a
  WHERE a.PatientID = p.PatientID
   AND a.AppointmentDate >= CURDATE() - INTERVAL 3 YEAR
);
```

UPDATE F. Give 5% Bonus to Doctors Who Generated > 10 Lakhs Revenue :

```
Doctors
SET Salary = Salary * 1.05
WHERE DoctorID IN (
  SELECT DoctorID
  FROM (
    SELECT d.DoctorID,
        SUM(b.BillAmount) AS TotalRevenue
    FROM Bills b
    JOIN Appointments a ON b.AppointmentID = a.AppointmentID
    JOIN Doctors d ON a.DoctorID = d.DoctorID
    GROUP BY d.DoctorID
    HAVING SUM(b.BillAmount) > 1000000
  ) AS RevenueDoctors
);
```

PROJECT 2 — Online Marketplace with Seller Performance & Fraud Detection :

1. Users (Buyers + Sellers)

```
CREATE TABLE Users (
   UserID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   UserType VARCHAR(20) CHECK (UserType IN ('Buyer','Seller')),
   CreatedAt DATE DEFAULT CURRENT_DATE
);
```

2. Categories :

```
CREATE TABLE Categories (
   CategoryID INT PRIMARY KEY,
   CategoryName VARCHAR(100) NOT NULL
);
```

3. Products :

```
CREATE TABLE Products (
   ProductID INT PRIMARY KEY,
   SellerID INT,
   CategoryID INT,
   ProductName VARCHAR(150) NOT NULL,
   Price DECIMAL(12,2) CHECK (Price > 0),
   StockQty INT CHECK (StockQty >= 0),
   FOREIGN KEY (SellerID) REFERENCES Users(UserID),
   FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);
```

4. Orders :

```
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
   BuyerID INT,
   OrderDate DATE NOT NULL,
```

```
    Status VARCHAR(30),
    FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);
```

5. OrderDetails (ON DELETE CASCADE) :

```
CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT CHECK (Quantity > 0),
    Price DECIMAL(12,2),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

6. Payments :

```
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY,
    OrderID INT,
    PaymentDate DATE,
    Amount DECIMAL(12,2),
    PaymentStatus VARCHAR(30), -- Paid / Refunded
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
```

7. Reviews :

```
CREATE TABLE Reviews (
    ReviewID INT PRIMARY KEY,
    ProductID INT,
    BuyerID INT,
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    ReviewDate DATE,
```

```
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID),
    FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);
```

A. Seller Performance Dashboard :

```
SELECT
    u.Name AS SellerName,
    COUNT(DISTINCT p.ProductID) AS TotalProducts,
    COUNT(DISTINCT o.OrderID) AS TotalOrders,
    SUM(od.Quantity * od.Price) AS TotalRevenue,
    AVG(r.Rating) AS AverageRating,
    CASE
        WHEN SUM(od.Quantity * od.Price) > 500000 THEN 'Platinum'
        WHEN SUM(od.Quantity * od.Price) > 200000 THEN 'Gold'
        ELSE 'Silver'
    END AS PerformanceLevel
FROM Users u
LEFT JOIN Products p ON u.UserID = p.SellerID
LEFT JOIN OrderDetails od ON p.ProductID = od.ProductID
LEFT JOIN Orders o ON od.OrderID = o.OrderID
LEFT JOIN Reviews r ON p.ProductID = r.ProductID
WHERE u.UserType = 'Seller'
GROUP BY u.UserID, u.Name;
```

More than 5 orders in one day :

```
SELECT BuyerID, OrderDate, COUNT(*) AS OrderCount
FROM Orders
GROUP BY BuyerID, OrderDate
HAVING COUNT(*) > 5;
```

Refund Rate > 50% :

```
SELECT o.BuyerID
```

```
FROM Orders o
JOIN Payments p ON o.OrderID = p.OrderID
GROUP BY o.BuyerID
HAVING
SUM(CASE WHEN p.PaymentStatus = 'Refunded' THEN 1 ELSE 0 END) * 1.0
/
COUNT(*) > 0.5;
```

Most Profitable Category :

```
SELECT c.CategoryName,
    SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Categories c
JOIN Products p ON c.CategoryID = p.CategoryID
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY c.CategoryName
ORDER BY TotalRevenue DESC
LIMIT 1;
```

Delete Inactive Sellers :

```
DELETE FROM Users u
WHERE u.UserType = 'Seller'
AND (
  NOT EXISTS (
    SELECT 1 FROM Products p
    WHERE p.SellerID = u.UserID
  )
  OR
  NOT EXISTS (
    SELECT 1
    FROM Products p
    JOIN OrderDetails od ON p.ProductID = od.ProductID
```

```
      JOIN Orders o ON od.OrderID = o.OrderID
      WHERE p.SellerID = u.UserID
        AND o.OrderDate >= CURDATE() - INTERVAL 1 YEAR
   )
);
```

Apply 10% Discount to Slow-Moving Products :

```
UPDATE Products p
SET Price = Price * 0.90
WHERE (
   SELECT IFNULL(SUM(od.Quantity),0)
   FROM OrderDetails od
   WHERE od.ProductID = p.ProductID
) <
(
   SELECT AVG(ProductQty)
   FROM (
      SELECT SUM(od2.Quantity) AS ProductQty
      FROM Products p2
      JOIN OrderDetails od2 ON p2.ProductID = od2.ProductID
      WHERE p2.CategoryID = p.CategoryID
      GROUP BY p2.ProductID
   ) AS CategoryAvg
);
```

Find Products Never Reviewed :

```
SELECT p.ProductID, p.ProductName
FROM Products p
WHERE NOT EXISTS (
   SELECT 1
   FROM Reviews r
```

```
    WHERE r.ProductID = p.ProductID
);
```

PROJECT 3 - "Global Ride-Sharing Analytics & Compliance System" :

Cities :

```
CREATE TABLE Cities (
    CityID INT PRIMARY KEY,
    CityName VARCHAR(100) UNIQUE
);
```

Drivers :

```
CREATE TABLE Drivers (
    DriverID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) UNIQUE,
    CityID INT,
    JoinDate DATE,
    FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);
```

Riders :

```
CREATE TABLE Riders (
    RiderID INT PRIMARY KEY,
    Name VARCHAR(100),
    Phone VARCHAR(15) UNIQUE
);
```

Vehicles :

```
CREATE TABLE Vehicles (
    VehicleID INT PRIMARY KEY,
    DriverID INT,
    Type VARCHAR(50),
    PlateNumber VARCHAR(20) UNIQUE,
```

```
    FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID)
);
```

Trips :

```
CREATE TABLE Trips (
    TripID INT PRIMARY KEY,
    DriverID INT,
    RiderID INT,
    CityID INT,
    TripDate DATE,
    DistanceKM DECIMAL(8,2) CHECK (DistanceKM > 0),
    FareAmount DECIMAL(10,2) CHECK (FareAmount > 0),
    FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID),
    FOREIGN KEY (RiderID) REFERENCES Riders(RiderID),
    FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);
```

Payments :

```
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY,
    TripID INT,
    PaymentMethod VARCHAR(20),
    PaidAmount DECIMAL(10,2),
    FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);
```

DriverRatings :

```
CREATE TABLE DriverRatings (
    RatingID INT PRIMARY KEY,
    TripID INT,
    RatingScore INT CHECK (RatingScore BETWEEN 1 AND 5),
    FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
```

);

Detect Underperforming Drivers :

```
SELECT
   d.Name,
   c.CityName,
   AVG(r.RatingScore) AS AvgRating,
   COUNT(t.TripID) AS TotalTrips,
   CASE
      WHEN AVG(r.RatingScore) <
          (SELECT AVG(r2.RatingScore)
           FROM Trips t2
           JOIN DriverRatings r2 ON t2.TripID = r2.TripID
           WHERE t2.CityID = d.CityID)
       AND COUNT(t.TripID) <
          (SELECT AVG(TripCount)
           FROM (
               SELECT COUNT(*) AS TripCount
               FROM Trips
               WHERE CityID = d.CityID
               GROUP BY DriverID
           ) AS CityTrips)
      THEN 'Underperforming'
      WHEN AVG(r.RatingScore) >= 4.5 THEN 'Excellent'
      ELSE 'Stable'
   END AS Status
FROM Drivers d
JOIN Cities c ON d.CityID = c.CityID
LEFT JOIN Trips t ON d.DriverID = t.DriverID
LEFT JOIN DriverRatings r ON t.TripID = r.TripID
```

```
GROUP BY d.DriverID, d.Name, c.CityName;
```

Revenue Consistency Analysis :

```
SELECT d.Name
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.DriverID, d.Name
HAVING COUNT(*) >= 5
AND MAX(t.FareAmount) > 3 * AVG(t.FareAmount);
```

City Revenue Imbalance :

```
SELECT c.CityName
FROM Cities c
JOIN Trips t ON c.CityID = t.CityID
GROUP BY c.CityID, c.CityName
HAVING
(
   SELECT SUM(FareAmount)
   FROM Trips t2
   WHERE t2.CityID = c.CityID
    AND t2.DriverID IN (
       SELECT DriverID
       FROM Trips
       WHERE CityID = c.CityID
       GROUP BY DriverID
       ORDER BY SUM(FareAmount) DESC
       LIMIT 1   -- Approx top segment
    )
)
>
0.6 * SUM(t.FareAmount);
```

Riders Using Multiple Cities Same Day :

```
SELECT RiderID, TripDate
FROM Trips
GROUP BY RiderID, TripDate
HAVING COUNT(DISTINCT CityID) > 1;
```

Driver Loyalty Bonus :

```
UPDATE Trips t
SET FareAmount = FareAmount * 1.05
WHERE t.DriverID IN (
   SELECT d.DriverID
   FROM Drivers d
   WHERE
      (SELECT COUNT(*)
       FROM Trips
       WHERE DriverID = d.DriverID)
      >
      (SELECT AVG(TripCount)
       FROM (
           SELECT COUNT(*) AS TripCount
           FROM Trips
           WHERE CityID = d.CityID
           GROUP BY DriverID
       ) AS CityAvg)
   AND
      (SELECT AVG(r.RatingScore)
       FROM Trips t2
       JOIN DriverRatings r ON t2.TripID = r.TripID
       WHERE t2.DriverID = d.DriverID)
      >
```

```
    (SELECT AVG(RatingScore) FROM DriverRatings)
);
```

Delete Inactive Drivers :

```
DELETE FROM Drivers d
WHERE NOT EXISTS (
    SELECT 1 FROM Trips t
    WHERE t.DriverID = d.DriverID
)
OR NOT EXISTS (
    SELECT 1 FROM Trips t
    WHERE t.DriverID = d.DriverID
    AND t.TripDate >= CURDATE() - INTERVAL 2 YEAR
);
```

Fraud Detection Label :

```
SELECT
  t.TripID,
  d.Name,
  c.CityName,
  CASE
    WHEN t.DistanceKM < 1
      AND t.FareAmount >
        (SELECT AVG(FareAmount)
         FROM Trips
         WHERE CityID = t.CityID)
    THEN 'Suspicious'
    WHEN p.PaymentMethod = 'Cash'
      AND t.FareAmount > 5000
    THEN 'High Risk'
    ELSE 'Normal'
```

```
    END AS RiskLevel
FROM Trips t
JOIN Drivers d ON t.DriverID = d.DriverID
JOIN Cities c ON t.CityID = c.CityID
JOIN Payments p ON t.TripID = p.TripID;
```

Multi-Level Business Metric :

```
SELECT c.CityName
FROM Cities c
JOIN Trips t ON c.CityID = t.CityID
GROUP BY c.CityID, c.CityName
HAVING
AVG(t.FareAmount / t.DistanceKM)
>
(SELECT AVG(FareAmount / DistanceKM) FROM Trips)
AND
COUNT(*) >
(SELECT AVG(CityTrips)
 FROM (
     SELECT COUNT(*) AS CityTrips
     FROM Trips
     GROUP BY CityID
 ) AS AvgTrips);
```

Set Operator Challenge :

A = Rating ≥ 4

```
SELECT DISTINCT d.DriverID
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
```

```
WHERE r.RatingScore >= 4
```

B = Revenue above city average

```
SELECT DriverID
FROM Trips t
GROUP BY DriverID, CityID
HAVING SUM(FareAmount) >
(
   SELECT AVG(CityRevenue)
   FROM (
      SELECT SUM(FareAmount) AS CityRevenue
      FROM Trips
      WHERE CityID = t.CityID
      GROUP BY DriverID
   ) AS CityAvg
)
-- Both conditions
A INTERSECT B;


-- High revenue but low rating
B EXCEPT A;


-- All strong drivers
A UNION B;
```

Advanced Integrity Validation Query :

```
SELECT t.TripID
FROM Trips t
JOIN Payments p ON t.TripID = p.TripID
JOIN Drivers d ON t.DriverID = d.DriverID
WHERE p.PaidAmount <> t.FareAmount
```

```
  OR d.CityID <> t.CityID;
```

PROJECT 1 — Smart Hospital Management & Billing Analytics System

DATA BASE DESIGNS : PATIENTS :

```
CREATE TABLE Patients (
   PatientID INT PRIMARY KEY,
   PatientName VARCHAR(100) NOT NULL,
   Phone VARCHAR(15) UNIQUE,
   Email VARCHAR(100) UNIQUE,
   DateOfBirth DATE,
   CreatedAt DATE DEFAULT CURRENT_DATE
);
```

2. SPECIALIZATIONS :

```
CREATE TABLE Specializations (
   SpecializationID INT PRIMARY KEY,
   SpecializationName VARCHAR(100) NOT NULL
);
```

3. DOCTORS :

```
CREATE TABLE Doctors (
   DoctorID INT PRIMARY KEY,
   DoctorName VARCHAR(100) NOT NULL,
   SpecializationID INT,
   Salary DECIMAL(12,2),
   FOREIGN KEY (SpecializationID) REFERENCES Specializations(SpecializationID)
);
```

4. APPOINTMENTS :

```
CREATE TABLE Appointments (
```

```
    AppointmentID INT PRIMARY KEY,
    PatientID INT,
    DoctorID INT,
    AppointmentDate DATE NOT NULL,
    Status VARCHAR(20),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);
```

5. TREATMENTS :

```
CREATE TABLE Treatments (
    TreatmentID INT PRIMARY KEY,
    TreatmentName VARCHAR(100) NOT NULL,
    Cost DECIMAL(10,2) CHECK (Cost > 0)
);
```

6. INSURANCE PROVIDERS :

```
CREATE TABLE InsuranceProviders (
    InsuranceID INT PRIMARY KEY,
    ProviderName VARCHAR(100) NOT NULL,
    CoveragePercent DECIMAL(5,2)
);
```

7. BILLS :

```
CREATE TABLE Bills (
    BillID INT PRIMARY KEY,
    AppointmentID INT,
    TreatmentID INT,
    InsuranceID INT,
    BillAmount DECIMAL(12,2) CHECK (BillAmount > 0),
    InsuranceCoveredAmount DECIMAL(12,2) DEFAULT 0,
    PaymentStatus VARCHAR(20),
```

```
    BillDate DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID),
    FOREIGN KEY (InsuranceID) REFERENCES InsuranceProviders(InsuranceID)
);
```

- A. Patient Billing Report :

```
SELECT
    p.PatientName,
    d.DoctorName,
    s.SpecializationName,
    t.TreatmentName,
    a.AppointmentDate,
    b.BillAmount,
    b.InsuranceCoveredAmount,
    (b.BillAmount - b.InsuranceCoveredAmount) AS FinalPayableAmount
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecializationID = s.SpecializationID
JOIN Treatments t ON b.TreatmentID = t.TreatmentID;
```

- B. Top 3 Revenue Generating Doctors:

```
SELECT d.DoctorName,
      SUM(b.BillAmount) AS TotalRevenue
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Doctors d ON a.DoctorID = d.DoctorID
GROUP BY d.DoctorName
ORDER BY TotalRevenue DESC
```

```
LIMIT 3;
```

- C. Identify Patients with Pending Payments:

```
SELECT
   p.PatientName,
   (b.BillAmount - b.InsuranceCoveredAmount) AS FinalPayableAmount,
   CASE
      WHEN (b.BillAmount - b.InsuranceCoveredAmount) > 0 THEN 'Pending'
      ELSE 'Paid'
   END AS PaymentStatus
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID;
```

- D. Specializations Generating Revenue Above Average :

```
SELECT s.SpecializationName,
     SUM(b.BillAmount) AS TotalRevenue
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecializationID = s.SpecializationID
GROUP BY s.SpecializationName
HAVING SUM(b.BillAmount) >
    (SELECT AVG(TotalRevenue)
     FROM (
        SELECT SUM(b2.BillAmount) AS TotalRevenue
        FROM Bills b2
        JOIN Appointments a2 ON b2.AppointmentID = a2.AppointmentID
        JOIN Doctors d2 ON a2.DoctorID = d2.DoctorID
        GROUP BY d2.SpecializationID
     ) AS RevenueTable);
```

🔹 E. Delete Inactive Patients (No Appointments in Last 3 Years) :

```
DELETE FROM Patients p
WHERE NOT EXISTS (
   SELECT 1
   FROM Appointments a
   WHERE a.PatientID = p.PatientID
     AND a.AppointmentDate >= CURDATE() - INTERVAL 3 YEAR
);
```

🔹 F. Give 5% Bonus to Doctors Who Generated > 10 Lakhs Revenue :

```
UPDATE Doctors
SET Salary = Salary * 1.05
WHERE DoctorID IN (
   SELECT DoctorID
   FROM (
      SELECT d.DoctorID,
            SUM(b.BillAmount) AS TotalRevenue
      FROM Bills b
      JOIN Appointments a ON b.AppointmentID = a.AppointmentID
      JOIN Doctors d ON a.DoctorID = d.DoctorID
      GROUP BY d.DoctorID
      HAVING SUM(b.BillAmount) > 1000000
   ) AS RevenueDoctors
);
```

PROJECT 2 — Online Marketplace with Seller Performance & Fraud Detection :

🔹 1. Users (Buyers + Sellers)

```
CREATE TABLE Users (
   UserID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Email VARCHAR(100) UNIQUE,
```

```
    UserType VARCHAR(20) CHECK (UserType IN ('Buyer','Seller')),
    CreatedAt DATE DEFAULT CURRENT_DATE
);
```

- 2. Categories :

```
CREATE TABLE Categories (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(100) NOT NULL
);
```

- 3. Products :

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    SellerID INT,
    CategoryID INT,
    ProductName VARCHAR(150) NOT NULL,
    Price DECIMAL(12,2) CHECK (Price > 0),
    StockQty INT CHECK (StockQty >= 0),
    FOREIGN KEY (SellerID) REFERENCES Users(UserID),
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);
```

- 4. Orders :

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    BuyerID INT,
    OrderDate DATE NOT NULL,
    Status VARCHAR(30),
    FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);
```

- 5. OrderDetails (ON DELETE CASCADE) :

```
CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT CHECK (Quantity > 0),
    Price DECIMAL(12,2),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

- 6. Payments :

```
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY,
    OrderID INT,
    PaymentDate DATE,
    Amount DECIMAL(12,2),
    PaymentStatus VARCHAR(30), -- Paid / Refunded
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
```

- 7. Reviews :

```
CREATE TABLE Reviews (
    ReviewID INT PRIMARY KEY,
    ProductID INT,
    BuyerID INT,
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    ReviewDate DATE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID),
    FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);
```

- A. Seller Performance Dashboard :

```
SELECT
    u.Name AS SellerName,
    COUNT(DISTINCT p.ProductID) AS TotalProducts,
    COUNT(DISTINCT o.OrderID) AS TotalOrders,
    SUM(od.Quantity * od.Price) AS TotalRevenue,
    AVG(r.Rating) AS AverageRating,
    CASE
        WHEN SUM(od.Quantity * od.Price) > 500000 THEN 'Platinum'
        WHEN SUM(od.Quantity * od.Price) > 200000 THEN 'Gold'
        ELSE 'Silver'
    END AS PerformanceLevel
FROM Users u
LEFT JOIN Products p ON u.UserID = p.SellerID
LEFT JOIN OrderDetails od ON p.ProductID = od.ProductID
LEFT JOIN Orders o ON od.OrderID = o.OrderID
LEFT JOIN Reviews r ON p.ProductID = r.ProductID
WHERE u.UserType = 'Seller'
GROUP BY u.UserID, u.Name;
```

More than 5 orders in one day :

```
SELECT BuyerID, OrderDate, COUNT(*) AS OrderCount
FROM Orders
GROUP BY BuyerID, OrderDate
HAVING COUNT(*) > 5;
```

Refund Rate > 50% :

```
SELECT o.BuyerID
FROM Orders o
JOIN Payments p ON o.OrderID = p.OrderID
GROUP BY o.BuyerID
HAVING
```

```
SUM(CASE WHEN p.PaymentStatus = 'Refunded' THEN 1 ELSE 0 END) * 1.0
/
COUNT(*) > 0.5;
```

Most Profitable Category :

```
SELECT c.CategoryName,
      SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Categories c
JOIN Products p ON c.CategoryID = p.CategoryID
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY c.CategoryName
ORDER BY TotalRevenue DESC
LIMIT 1;
```

Delete Inactive Sellers :

```
DELETE FROM Users u
WHERE u.UserType = 'Seller'
AND (
   NOT EXISTS (
      SELECT 1 FROM Products p
      WHERE p.SellerID = u.UserID
   )
   OR
   NOT EXISTS (
      SELECT 1
      FROM Products p
      JOIN OrderDetails od ON p.ProductID = od.ProductID
      JOIN Orders o ON od.OrderID = o.OrderID
      WHERE p.SellerID = u.UserID
        AND o.OrderDate >= CURDATE() - INTERVAL 1 YEAR
   )
```

```
);
```

Apply 10% Discount to Slow-Moving Products :

```
UPDATE Products p
SET Price = Price * 0.90
WHERE (
   SELECT IFNULL(SUM(od.Quantity),0)
   FROM OrderDetails od
   WHERE od.ProductID = p.ProductID
) <
(
   SELECT AVG(ProductQty)
   FROM (
      SELECT SUM(od2.Quantity) AS ProductQty
      FROM Products p2
      JOIN OrderDetails od2 ON p2.ProductID = od2.ProductID
      WHERE p2.CategoryID = p.CategoryID
      GROUP BY p2.ProductID
   ) AS CategoryAvg
);
```

Find Products Never Reviewed :

```
SELECT p.ProductID, p.ProductName
FROM Products p
WHERE NOT EXISTS (
   SELECT 1
   FROM Reviews r
   WHERE r.ProductID = p.ProductID
);
```

PROJECT 3 - “Global Ride-Sharing Analytics & Compliance System” :

- Cities :

```
CREATE TABLE Cities (
    CityID INT PRIMARY KEY,
    CityName VARCHAR(100) UNIQUE
);
```

- Drivers :

```
CREATE TABLE Drivers (
    DriverID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) UNIQUE,
    CityID INT,
    JoinDate DATE,
    FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);
```

- Riders :

```
CREATE TABLE Riders (
    RiderID INT PRIMARY KEY,
    Name VARCHAR(100),
    Phone VARCHAR(15) UNIQUE
);
```

- Vehicles :

```
CREATE TABLE Vehicles (
    VehicleID INT PRIMARY KEY,
    DriverID INT,
    Type VARCHAR(50),
    PlateNumber VARCHAR(20) UNIQUE,
    FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID)
);
```

- Trips :

```
CREATE TABLE Trips (
    TripID INT PRIMARY KEY,
    DriverID INT,
    RiderID INT,
    CityID INT,
    TripDate DATE,
    DistanceKM DECIMAL(8,2) CHECK (DistanceKM > 0),
    FareAmount DECIMAL(10,2) CHECK (FareAmount > 0),
    FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID),
    FOREIGN KEY (RiderID) REFERENCES Riders(RiderID),
    FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);
```

- Payments :

```
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY,
    TripID INT,
    PaymentMethod VARCHAR(20),
    PaidAmount DECIMAL(10,2),
    FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);
```

- DriverRatings :

```
CREATE TABLE DriverRatings (
    RatingID INT PRIMARY KEY,
    TripID INT,
    RatingScore INT CHECK (RatingScore BETWEEN 1 AND 5),
    FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);
```

Detect Underperforming Drivers :

```
SELECT
```

```
    d.Name,
    c.CityName,
    AVG(r.RatingScore) AS AvgRating,
    COUNT(t.TripID) AS TotalTrips,
    CASE
        WHEN AVG(r.RatingScore) <
            (SELECT AVG(r2.RatingScore)
             FROM Trips t2
             JOIN DriverRatings r2 ON t2.TripID = r2.TripID
             WHERE t2.CityID = d.CityID)
         AND COUNT(t.TripID) <
            (SELECT AVG(TripCount)
             FROM (
                 SELECT COUNT(*) AS TripCount
                 FROM Trips
                 WHERE CityID = d.CityID
                 GROUP BY DriverID
             ) AS CityTrips)
        THEN 'Underperforming'
        WHEN AVG(r.RatingScore) >= 4.5 THEN 'Excellent'
        ELSE 'Stable'
    END AS Status
FROM Drivers d
JOIN Cities c ON d.CityID = c.CityID
LEFT JOIN Trips t ON d.DriverID = t.DriverID
LEFT JOIN DriverRatings r ON t.TripID = r.TripID
GROUP BY d.DriverID, d.Name, c.CityName;
```

Revenue Consistency Analysis :

```
SELECT d.Name
```

```
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.DriverID, d.Name
HAVING COUNT(*) >= 5
AND MAX(t.FareAmount) > 3 * AVG(t.FareAmount);
```

City Revenue Imbalance :

```
SELECT c.CityName
FROM Cities c
JOIN Trips t ON c.CityID = t.CityID
GROUP BY c.CityID, c.CityName
HAVING
(
   SELECT SUM(FareAmount)
   FROM Trips t2
   WHERE t2.CityID = c.CityID
    AND t2.DriverID IN (
       SELECT DriverID
       FROM Trips
       WHERE CityID = c.CityID
       GROUP BY DriverID
       ORDER BY SUM(FareAmount) DESC
       LIMIT 1   -- Approx top segment
    )
)
>
0.6 * SUM(t.FareAmount);
```

Riders Using Multiple Cities Same Day :

```
SELECT RiderID, TripDate
FROM Trips
```

```
GROUP BY RiderID, TripDate
HAVING COUNT(DISTINCT CityID) > 1;
```

Driver Loyalty Bonus :

```
UPDATE Trips t
SET FareAmount = FareAmount * 1.05
WHERE t.DriverID IN (
   SELECT d.DriverID
   FROM Drivers d
   WHERE
      (SELECT COUNT(*)
       FROM Trips
       WHERE DriverID = d.DriverID)
      >
      (SELECT AVG(TripCount)
       FROM (
           SELECT COUNT(*) AS TripCount
           FROM Trips
           WHERE CityID = d.CityID
           GROUP BY DriverID
       ) AS CityAvg)
   AND
      (SELECT AVG(r.RatingScore)
       FROM Trips t2
       JOIN DriverRatings r ON t2.TripID = r.TripID
       WHERE t2.DriverID = d.DriverID)
      >
      (SELECT AVG(RatingScore) FROM DriverRatings)
);
```

Delete Inactive Drivers :

```
DELETE FROM Drivers d
WHERE NOT EXISTS (
    SELECT 1 FROM Trips t
    WHERE t.DriverID = d.DriverID
)
OR NOT EXISTS (
    SELECT 1 FROM Trips t
    WHERE t.DriverID = d.DriverID
    AND t.TripDate >= CURDATE() - INTERVAL 2 YEAR
);
```

Fraud Detection Label :

```
SELECT
  t.TripID,
  d.Name,
  c.CityName,
  CASE
    WHEN t.DistanceKM < 1
      AND t.FareAmount >
        (SELECT AVG(FareAmount)
         FROM Trips
         WHERE CityID = t.CityID)
    THEN 'Suspicious'
    WHEN p.PaymentMethod = 'Cash'
      AND t.FareAmount > 5000
    THEN 'High Risk'
    ELSE 'Normal'
  END AS RiskLevel
FROM Trips t
JOIN Drivers d ON t.DriverID = d.DriverID
```

```
JOIN Cities c ON t.CityID = c.CityID
JOIN Payments p ON t.TripID = p.TripID;
```

Multi-Level Business Metric :

```
SELECT c.CityName
FROM Cities c
JOIN Trips t ON c.CityID = t.CityID
GROUP BY c.CityID, c.CityName
HAVING
AVG(t.FareAmount / t.DistanceKM)
>
(SELECT AVG(FareAmount / DistanceKM) FROM Trips)
AND
COUNT(*) >
(SELECT AVG(CityTrips)
 FROM (
     SELECT COUNT(*) AS CityTrips
     FROM Trips
     GROUP BY CityID
 ) AS AvgTrips);
```

Set Operator Challenge :

A = Rating ≥ 4

```
SELECT DISTINCT d.DriverID
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
WHERE r.RatingScore >= 4
```

B = Revenue above city average

```
SELECT DriverID
```

```
FROM Trips t
GROUP BY DriverID, CityID
HAVING SUM(FareAmount) >
(
   SELECT AVG(CityRevenue)
   FROM (
      SELECT SUM(FareAmount) AS CityRevenue
      FROM Trips
      WHERE CityID = t.CityID
      GROUP BY DriverID
   ) AS CityAvg
)
-- Both conditions
A INTERSECT B;

-- High revenue but low rating
B EXCEPT A;

-- All strong drivers
A UNION B;
```

Advanced Integrity Validation Query :

```
SELECT t.TripID
FROM Trips t
JOIN Payments p ON t.TripID = p.TripID
JOIN Drivers d ON t.DriverID = d.DriverID
WHERE p.PaidAmount <> t.FareAmount
   OR d.CityID <> t.CityID;
```

Date:-26/02/2026 ---- 27/02/2026

HTML , CSS & JAVA SCRIPT PROJECTS QUESTIONS :

Project 1 — “Smart College Event Registration Portal” :

Index.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Smart College Event Portal</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
   <h1>Smart College Event Registration Portal</h1>
</header>

<nav>
   <a href="index.html">Home</a> |
   <a href="events.html">Events</a> |
   <a href="register.html">Register</a>
</nav>
```

```
<section>
  <article>
    <h2>Welcome Students!</h2>
    <p>Register for upcoming college events easily.</p>
  </article>

  <article>
    <h3>Event Schedule</h3>
    <ol>
      <li>Tech Fest - March 10</li>
      <li>Cultural Fest - March 15</li>
      <li>Sports Meet - March 20</li>
    </ol>
  </article>
</section>

<footer>
  <p>© 2026 Smart College</p>
</footer>

<script src="script.js"></script>
</body>
</html>
```

Events .html :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
    <title>Events</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    <h1>Upcoming Events</h1>
</header>

<nav>
    <a href="index.html">Home</a> >
    <a href="events.html">Events</a>
</nav>

<section>
    <article>
        <table border="1">
            <tr>
                <th>Event</th>
                <th>Date</th>
                <th>Venue</th>
                <th>Seats Left</th>
            </tr>
            <tr>
                <td>Tech Fest</td>
                <td>10-03-2026</td>
                <td>Auditorium</td>
                <td><meter value="30" min="0" max="100"></meter></td>
            </tr>
```

```
        <tr>
          <td>Cultural Fest</td>
          <td>15-03-2026</td>
          <td>Main Ground</td>
          <td><meter value="50" min="0" max="100"></meter></td>
        </tr>
      </table>
    </article>

    <article>
      <h3>Rules</h3>
      <ul>
        <li>ID card mandatory</li>
        <li>No late entries</li>
        <li>Maintain discipline</li>
      </ul>
    </article>
</section>

<footer>
    <p>Contact: college@email.com</p>
</footer>

</body>
</html>
```

Register.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Register</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    <h1>Event Registration</h1>
</header>

<nav>
    <a href="index.html">Home</a> >
    <a href="register.html">Register</a>
</nav>

<section>

<form id="regForm">

<label>Name:</label>
<input type="text" id="name" required onchange="updateProgress()"><br><br>

<label>Email:</label>
<input type="email" id="email" required onchange="updateProgress()"><br><br>

<label>Age:</label>
<input type="number" id="age" min="18" max="30" required onchange="updateProgress()"><br><br>

<label>Date:</label>
```

```
<input type="date" required onchange="updateProgress()"><br><br>

<label>Department:</label>
<input list="departments" required onchange="updateProgress()">
<datalist id="departments">
<option value="CSE">
<option value="ECE">
<option value="EEE">
<option value="MBA">
</datalist>
<br><br>

<label>Gender:</label>
<input type="radio" name="gender"> Male
<input type="radio" name="gender"> Female
<br><br>

<label>Select Event:</label>
<input type="checkbox"> Tech Fest
<input type="checkbox"> Cultural Fest
<br><br>

<label>Form Completion:</label>
<progress id="progressBar" value="0" max="100"></progress>
<br><br>

<button type="button" onclick="register()">Submit</button>

</form>
```

```
<button onclick="getLocation()">Get Location</button>
<p id="location"></p>

<p id="message"></p>

</section>

<footer>
  <p>© 2026 Smart College</p>
</footer>

<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
  font-family: Arial;
  margin: 0;
  padding: 0;
}

header, footer {
  background-color: darkblue;
  color: white;
  text-align: center;
  padding: 15px;
}
```

```
nav {
    background-color: lightgray;
    padding: 10px;
    text-align: center;
}

section {
    display: flex;
    justify-content: space-around;
    padding: 20px;
}

article {
    border: 1px solid gray;
    padding: 15px;
    margin: 10px;
    width: 45%;
}

/* Responsive */
@media (max-width: 768px) {
    section {
        flex-direction: column;
    }

    article {
        width: 100%;
    }
}
```

Script.js :

```
// Update Progress Bar
function updateProgress() {
    let inputs = document.querySelectorAll("#regForm input[required]");
    let filled = 0;

    for (let i = 0; i < inputs.length; i++) {
        if (inputs[i].value !== "") {
            filled++;
        }
    }

    let percent = (filled / inputs.length) * 100;
    document.getElementById("progressBar").value = percent;
}

// Register Function
function register() {
    let name = document.getElementById("name").value;

    if (name === "") {
        alert("Name is required");
    } else {
        localStorage.setItem("studentName", name);
        document.getElementById("message").innerHTML =
            "Registration Successful! Welcome " + name;
    }
}
```

```
// Retrieve on Reload
window.onload = function () {
   let savedName = localStorage.getItem("studentName");
   if (savedName) {
      document.getElementById("message").innerHTML =
         "Welcome back " + savedName;
   }
};

// Geolocation
function getLocation() {
   if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(showPosition, showError);
   } else {
      alert("Geolocation not supported");
   }
}

function showPosition(position) {
   document.getElementById("location").innerHTML =
      "Latitude: " + position.coords.latitude +
      " Longitude: " + position.coords.longitude;
}

function showError(error) {
   document.getElementById("location").innerHTML =
      "Permission denied or location unavailable.";
}
```

Project 2 — “Personal Finance Tracker (Client-Side Only)” :

Index.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Personal Finance Tracker</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
   <h1>Personal Finance Tracker</h1>
</header>

<nav>
   <a href="#">Home</a>
</nav>

<section class="container">

   <!-- Form Section -->
   <article class="form-section">
      <h2>Add Transaction</h2>

      <form id="financeForm">

         <label>Description:</label>
         <input type="text" id="desc" required><br><br>
```

```
        <label>Amount:</label>
        <input type="number" id="amount" required><br><br>

        <label>Date:</label>
        <input type="date" id="date" required><br><br>

        <label>Category:</label>
        <input list="categories" id="category" required>
        <datalist id="categories">
          <option value="Salary">
          <option value="Food">
          <option value="Transport">
          <option value="Shopping">
          <option value="Bills">
        </datalist>
        <br><br>

        <label>Type:</label>
        <input type="radio" name="type" value="income" required> Income
        <input type="radio" name="type" value="expense" required> Expense
        <br><br>

        <button type="button" onclick="addTransaction()">Add</button>

    </form>
</article>

<!-- Table Section -->
<article class="table-section">
```

```
    <h2>Transactions</h2>

    <table border="1">
      <thead>
        <tr>
          <th>Description</th>
          <th>Amount</th>
          <th>Date</th>
          <th>Category</th>
          <th>Type</th>
        </tr>
      </thead>
      <tbody id="transactionTable">
      </tbody>
    </table>

    <h3>Total Balance: ₹ <span id="balance">0</span></h3>

    <h3>Savings Goal</h3>
    <progress id="savingsProgress" value="0" max="100000"></progress>

    <h3>Expense Ratio</h3>
    <meter id="expenseMeter" min="0" max="100" value="0"></meter>

  </article>

</section>
```

```
<section>
  <h3>Financial Tips</h3>
  <ol>
    <li>Track every expense.</li>
    <li>Save at least 20% of income.</li>
    <li>Avoid unnecessary subscriptions.</li>
  </ol>
</section>

<footer>
  <p>© 2026 Finance Tracker</p>
</footer>

<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
  font-family: Arial;
  margin: 0;
  padding: 0;
  background-color: #f4f6f9;
}

header, footer {
  background-color: #2c3e50;
  color: white;
  text-align: center;
  padding: 15px;
```

```
}

.container {
    display: flex;
    justify-content: space-around;
    padding: 20px;
}

article {
    background-color: white;
    padding: 20px;
    margin: 10px;
    border: 1px solid #ccc;
    width: 45%;
}

/* Hover effect */
button:hover {
    background-color: #27ae60;
    color: white;
    cursor: pointer;
}

/* Responsive */
@media (max-width: 768px) {
    .container {
        flex-direction: column;
    }
```

```
    article {
        width: 100%;
    }
}
```

Script.js :

```
// Array to store transactions
let transactions = JSON.parse(localStorage.getItem("transactions")) || [];

// Add Transaction
function addTransaction() {

    let desc = document.getElementById("desc").value;
    let amount = parseFloat(document.getElementById("amount").value);
    let date = document.getElementById("date").value;
    let category = document.getElementById("category").value;
    let type = document.querySelector('input[name="type"]:checked');

    if (!type) {
        alert("Select income or expense");
        return;
    }

    type = type.value;

    // Create object
    let transaction = {
        desc: desc,
        amount: amount,
        date: date,
```

```
        category: category,
        type: type
    };

    transactions.push(transaction);  // push()

    localStorage.setItem("transactions", JSON.stringify(transactions));

    updateTable();
    calculateTotal();

    document.getElementById("financeForm").reset();
}

// Update Table (DOM manipulation)
function updateTable() {

    let table = document.getElementById("transactionTable");
    table.innerHTML = "";

    transactions.map(function(t) {  // map()

        let row = table.insertRow();

        row.insertCell(0).innerHTML = t.desc;
        row.insertCell(1).innerHTML = t.amount;
        row.insertCell(2).innerHTML = t.date;
        row.insertCell(3).innerHTML = t.category;
        row.insertCell(4).innerHTML = t.type;
```

```
        // switch for category styling
        switch (t.category) {
            case "Food":
                row.style.backgroundColor = "#fdecea";
                break;
            case "Salary":
                row.style.backgroundColor = "#e8f8f5";
                break;
            default:
                row.style.backgroundColor = "#fdfefe";
        }

    });
}

// Calculate Totals
function calculateTotal() {

    let income = 0;
    let expense = 0;

    transactions.forEach(function(t) {   // loop
        if (t.type === "income") {
            income += t.amount;
        } else {
            expense += t.amount;
        }
    });
```

```
    let balance = income - expense;

    document.getElementById("balance").innerText = balance;

    // progress bar for savings
    document.getElementById("savingsProgress").value = balance;

    // expense ratio
    let ratio = income > 0 ? (expense / income) * 100 : 0;
    document.getElementById("expenseMeter").value = ratio;
}

// Load on refresh
window.onload = function() {
    updateTable();
    calculateTotal();
};
```

Project 3 — "Multi-Page E-Learning Platform" :

Dashboard.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Dashboard</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<header>
  <h1>E-Learning Platform</h1>
</header>

<nav>
  <a href="dashboard.html">Dashboard</a> |
  <a href="courses.html">Courses</a> |
  <a href="quiz.html">Quiz</a> |
  <a href="profile.html">Profile</a>
</nav>

<p>Home > Dashboard</p>

<section class="grid-dashboard">
  <div class="card">Total Courses: <span id="totalCourses"></span></div>
  <div class="card">Completed: <span id="completedCourses"></span></div>
  <div class="card">Last Score: <span id="lastScore"></span></div>
</section>

<script src="script.js"></script>
</body>
</html>
```

Courses.html :

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Courses</title>
<link rel="stylesheet" href="style.css">
```

```
</head>
<body>

<header><h1>Courses</h1></header>

<nav>
<a href="dashboard.html">Dashboard</a> >
<a href="courses.html">Courses</a>
</nav>

<section id="courseContainer" class="flex-courses"></section>

<h2>Course Table</h2>
<table border="1">
<tr>
<th>Course</th>
<th>Lessons</th>
<th>Status</th>
</tr>
<tbody id="courseTable"></tbody>
</table>

<script src="script.js"></script>
</body>
</html>
```

Quiz.html :

```
<!DOCTYPE html>
<html>
<head>
```

```
<meta charset="UTF-8">
<title>Quiz</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<header><h1>Quiz</h1></header>

<nav>
<a href="dashboard.html">Dashboard</a> >
<a href="quiz.html">Quiz</a>
</nav>

<section id="quizSection">
   <button onclick="loadQuiz()">Start Quiz</button>
   <div id="quizContainer"></div>
   <button onclick="submitQuiz()">Submit Quiz</button>
</section>

<section id="resultSection"></section>

<script src="script.js"></script>
</body>
</html>
```

Profile.html :

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
```

<title>Profile</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<header><h1>Profile</h1></header>

<nav>

Dashboard >

Profile

</nav>

<h2>Your Progress</h2>

<progress id="courseProgress" value="0" max="100"></progress>

<script src="script.js"></script>

</body>

</html>

Style.css :

body { font-family: Arial; margin:0; }

header { background:#2c3e50; color:white; padding:15px; }

nav { background:#ecf0f1; padding:10px; }

.grid-dashboard {

  display:grid;

```
    grid-template-columns: repeat(3, 1fr);
    gap:20px;
    padding:20px;
}

.card {
    background:#3498db;
    color:white;
    padding:20px;
}

.flex-courses {
    display:flex;
    gap:20px;
    padding:20px;
}

.flex-courses div {
    border:1px solid #ccc;
    padding:15px;
}

.flex-courses div:hover {
    background:#f1f1f1;
}

@media (max-width:768px) {
    .grid-dashboard { grid-template-columns:1fr; }
    .flex-courses { flex-direction:column; }
```

```
}
```

Script.js :

```
let courses = [
   { id:1, name:"HTML", lessons:["Intro","Tags","Forms"], completed:false },
   { id:2, name:"CSS", lessons:["Selectors","Flexbox","Grid"], completed:false },
   { id:3, name:"JavaScript", lessons:["Variables","DOM","Async"], completed:false }
];

let quizQuestions = [
   { q:"What is HTML?", options:["Language","Database","OS"], answer:0 },
   { q:"Which is JS framework?", options:["React","MySQL","Linux"], answer:0 }
];
```

Load Courses (DOM + Table + Ordered List) :

```
function loadCourses() {

   let container = document.getElementById("courseContainer");
   let table = document.getElementById("courseTable");

   if(!container) return;

   courses.forEach(course => {

      // Flex card
      let div = document.createElement("div");
      div.innerHTML = `
         <h3>${course.name}</h3>
         <ol>${course.lessons.map(l => `<li>${l}</li>`).join("")}</ol>
         <button onclick="completeCourse(${course.id})">Complete</button>
      `;
```

```
        container.appendChild(div);

        // Table
        let row = table.insertRow();
        row.insertCell(0).innerText = course.name;
        row.insertCell(1).innerText = course.lessons.length;
        row.insertCell(2).innerText = course.completed ? "Done" : "Pending";
    });
}
```

Async Simulation :

```
function fetchQuiz() {
    return new Promise(resolve => {
        setTimeout(() => resolve(quizQuestions), 1500);
    });
}

async function loadQuiz() {

    let container = document.getElementById("quizContainer");
    container.innerHTML = "Loading Quiz...";

    let questions = await fetchQuiz();

    container.innerHTML = "";

    questions.forEach((q,index) => {

        let div = document.createElement("div");
        div.innerHTML = `
```

```
        <p>${q.q}</p>
        ${q.options.map((opt,i)=>
          `<input type="radio" name="q${index}" value="${i}" onchange="handleSelect()"> ${opt}<br>`
        ).join("")}
      `;
      container.appendChild(div);
    });
}
function submitQuiz() {

    let score = 0;

    quizQuestions.forEach((q,index)=>{
      let selected = document.querySelector(`input[name="q${index}"]:checked`);
      if(selected && parseInt(selected.value) === q.answer){
        score++;
      }
    });

    let percent = (score/quizQuestions.length)*100;

    let grade;
    if(percent >= 80) grade = "A";
    else if(percent >= 50) grade = "B";
    else grade = "Fail";

    let feedback;
    switch(grade){
      case "A": feedback="Excellent!"; break;
```

```
        case "B": feedback="Good Job!"; break;
        default: feedback="Try Again!";
    }

    document.getElementById("resultSection").innerHTML =
        `Score: ${percent}% | Grade: ${grade} <br> ${feedback}`;

    localStorage.setItem("lastScore", percent);
}
function completeCourse(id){
    courses = courses.map(c=>{
        if(c.id===id) c.completed=true;
        return c;
    });

    localStorage.setItem("courses", JSON.stringify(courses));
    alert("Course Completed!");
}
window.onload = function(){

    if(localStorage.getItem("courses")){
        courses = JSON.parse(localStorage.getItem("courses"));
    }

    loadCourses();

    let lastScore = localStorage.getItem("lastScore") || 0;

    let total = courses.length;
```

```
  let completed = courses.filter(c=>c.completed).length;

  if(document.getElementById("totalCourses")){
    document.getElementById("totalCourses").innerText = total;
    document.getElementById("completedCourses").innerText = completed;
    document.getElementById("lastScore").innerText = lastScore + "%";
  }

  if(document.getElementById("courseProgress")){
    document.getElementById("courseProgress").value =
      (completed/total)*100;
  }
};
const { calculateGrade, calculatePercent } = require('./script');

test("Grade A", ()=>{
  expect(calculateGrade(85)).toBe("A");
});

test("Percent calculation", ()=>{
  expect(calculatePercent(2,2)).toBe(100);
});

test("Fail logic", ()=>{
  expect(calculateGrade(30)).toBe("Fail");
});
```

Project 4 — “Geo-Based Task Manager with Async Storage Simulation” :

Index.html :

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Geo Task Manager</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<header>
  <h1>Geo-Based Task Manager</h1>
</header>

<section class="grid-container">

  <!-- Task Form -->
  <article class="form-area">
    <h2>Add Task</h2>

    <form id="taskForm">

      <label>Task Title</label>
      <input type="text" id="title" required>

      <label>Due Date</label>
      <input type="date" id="date" required>

      <label>Estimated Hours</label>
      <input type="number" id="hours" min="1" required>
```

```
      <label>Priority</label>
      <input list="priorityList" id="priority" required>
      <datalist id="priorityList">
        <option value="Low">
        <option value="Medium">
        <option value="High">
      </datalist>

      <label>
        <input type="checkbox" id="completed">
        Completed
      </label>

      <button type="button" onclick="addTask()">Add Task</button>
    </form>
</article>

<!-- Task Table -->
<article class="table-area">
    <h2>Task List</h2>

    <table border="1">
      <thead>
        <tr>
          <th>Title</th>
          <th>Priority</th>
          <th>Location</th>
          <th>Status</th>
        </tr>
```

```
            </thead>
            <tbody id="taskTable"></tbody>
        </table>

        <h3>Completion Progress</h3>
        <progress id="taskProgress" value="0" max="100"></progress>

        <h3>High Priority Load</h3>
        <meter id="priorityMeter" min="0" max="100" value="0"></meter>

    </article>

</section>

<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
    font-family: Arial;
    margin: 0;
}

header {
    background: #1f2937;
    color: white;
    padding: 15px;
    text-align: center;
}
```

```
.grid-container {
    display: grid;
    grid-template-columns: 1fr 2fr;
    gap: 20px;
    padding: 20px;
}

.form-area, .table-area {
    background: #f9fafb;
    padding: 20px;
    border: 1px solid #ccc;
}

/* Advanced selector */
table tbody tr:nth-child(even) {
    background: #e5e7eb;
}

button:hover {
    background: #2563eb;
    color: white;
    cursor: pointer;
}

/* Responsive */
@media (max-width: 768px) {
    .grid-container {
        grid-template-columns: 1fr;
```

```
    }
}
```

Script.js :

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
async function addTask() {

    try {

        const location = await getLocation();

        const task = {
            id: Date.now(),
            title: document.getElementById("title").value,
            priority: document.getElementById("priority").value,
            location: location,
            completed: document.getElementById("completed").checked
        };

        await saveToServer(task);  // Async simulation

        tasks.push(task);         // push()
        localStorage.setItem("tasks", JSON.stringify(tasks));

        updateTable();
        updateStats();

        document.getElementById("taskForm").reset();

    } catch (error) {
```

```
      alert("Error: " + error);
    }
}
```

Geolocation API :

```
function getLocation() {

  return new Promise((resolve, reject) => {

    if (!navigator.geolocation) {
      reject("Geolocation not supported");
    }

    navigator.geolocation.getCurrentPosition(
      position => {
        resolve(
          position.coords.latitude.toFixed(4) +
          ", " +
          position.coords.longitude.toFixed(4)
        );
      },
      error => {
        switch(error.code) {
          case error.PERMISSION_DENIED:
            reject("Permission Denied");
            break;
          case error.TIMEOUT:
            reject("Timeout");
            break;
          default:
```

```
                reject("Unknown Error");
            }
        },
        { timeout: 5000 }
    );
  });
}
```

Async Server Simulation :

```
function saveToServer(task) {

  return new Promise((resolve, reject) => {

    setTimeout(() => {

      // Simulate random failure
      if (Math.random() < 0.9) {
        resolve("Saved Successfully");
      } else {
        reject("Server Error");
      }

    }, 1500);
  });
}
```

Update Table :

```
function updateTable() {

  const table = document.getElementById("taskTable");
  table.innerHTML = "";
```

```
    tasks.map(task => {

        const row = table.insertRow();

        row.insertCell(0).innerText = task.title;
        row.insertCell(1).innerText = task.priority;
        row.insertCell(2).innerText = task.location;
        row.insertCell(3).innerText = task.completed ? "Done" : "Pending";

    });
}
```

Update Progress :

```
function updateStats() {

    const completedTasks = tasks.filter(t => t.completed);
    const highPriority = tasks.filter(t => t.priority === "High");

    const completionPercent =
        tasks.length ? (completedTasks.length / tasks.length) * 100 : 0;

    const priorityPercent =
        tasks.length ? (highPriority.length / tasks.length) * 100 : 0;

    document.getElementById("taskProgress").value = completionPercent;
    document.getElementById("priorityMeter").value = priorityPercent;
}
window.onload = function () {
    updateTable();
```

```
    updateStats();
};
```